

มหาวิทยาลัยราชภัฏบุรีรัมย์ — BRU Strategic Tracking System

ชุดข้อมูลนำเสนอการทำงานโค้ด (Code Walkthrough Presentation Guide)

นำเสนอโค้ด

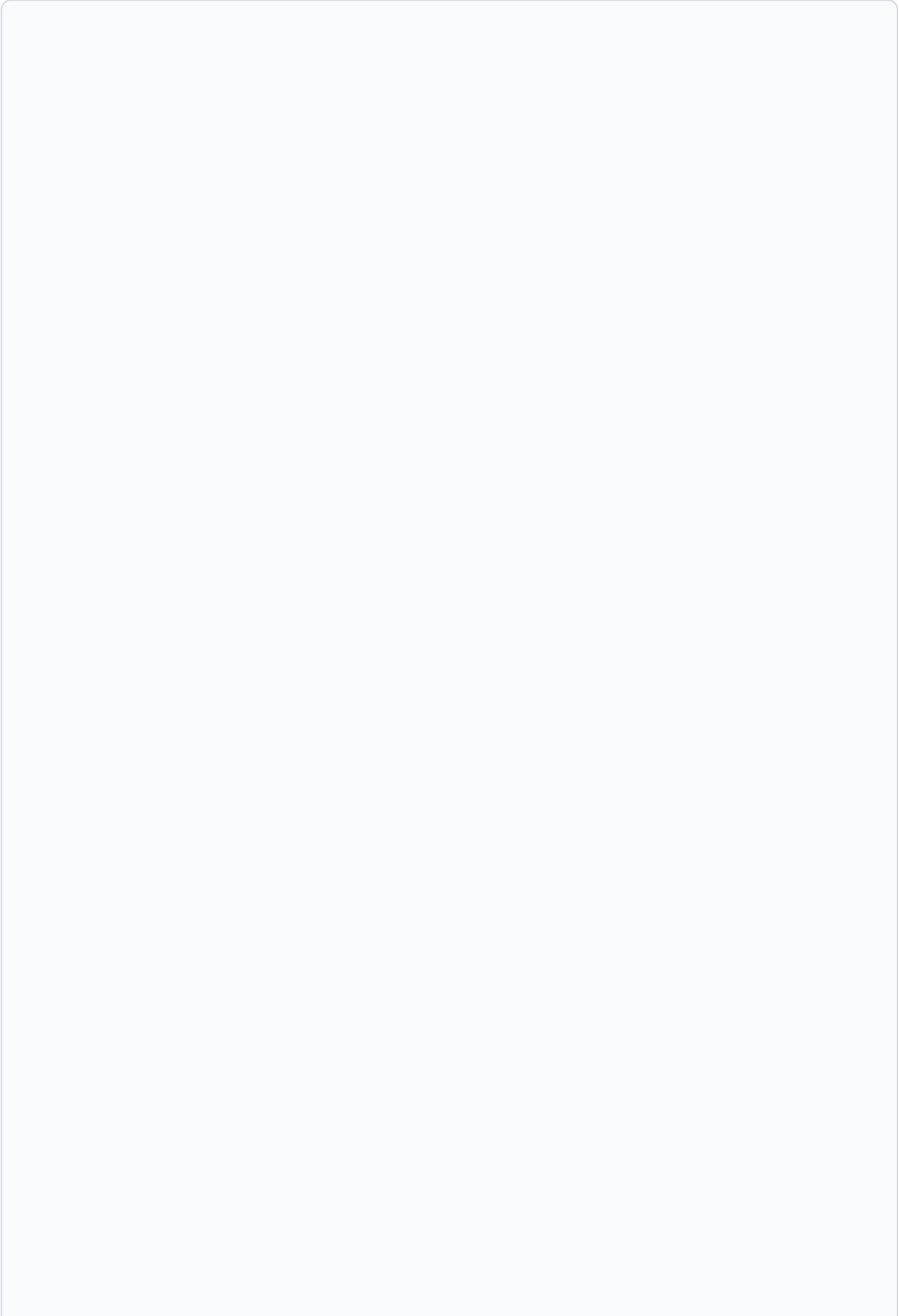
ฉบับสมบูรณ์

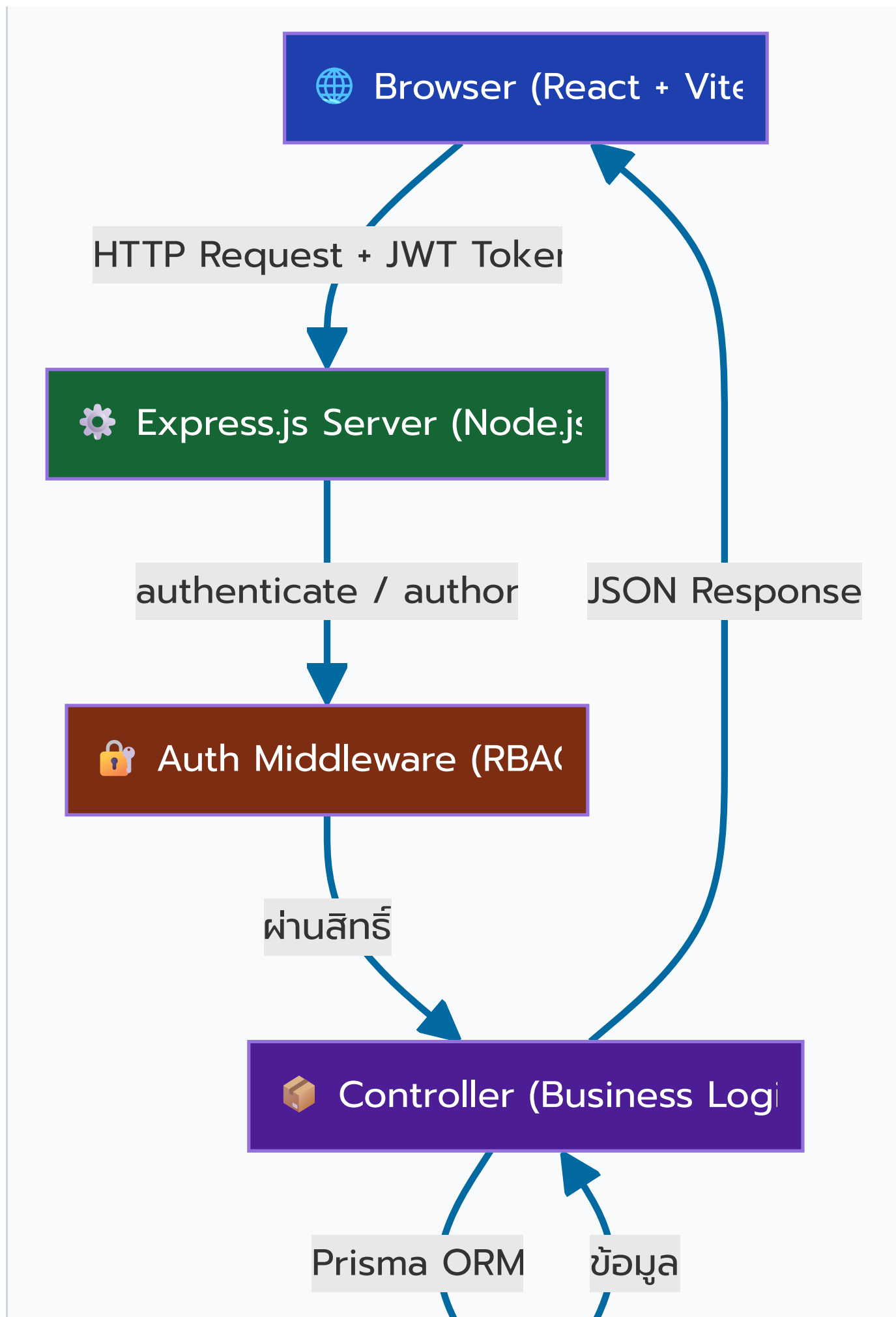
ชุดข้อมูลนำเสนอ: การทำงานโค้ดของระบบติดตามโครงการยุทธศาสตร์

BRU Strategic Tracking System — Code Walkthrough Guide



ภาพรวมสถาปัตยกรรมระบบ (System Architecture)







MySQL Database

1. 🚀 จุดเริ่มต้นเซิร์ฟเวอร์ — `backend/app.js`

หน้าที่: เป็นประตูทางเข้าหลักของระบบ Backend ทั้งหมด ตั้งค่าความปลอดภัย กำหนดเส้นทาง API และจัดการข้อผิดพลาด

โครงสร้างหลักของ `app.js` (11 ส่วน)

1. สร้างโฟลเดอร์ /uploads อัปเดตโมดูล (ถ้ายังไม่มี)
2. Helmet → เพิ่ม Security Headers ป้องกัน XSS, Clickjacking
3. CORS → กำหนด Origin ที่อนุญาต (localhost, Vercel, Netlify)
4. Rate Limit → ป้องกัน DDoS: ทัวไป 1,000 ครั้ง/15 นาที | Login 20 ครั้ง/15 นาที
5. Body Parser → รับข้อมูล JSON ขนาดสูงสุด 50MB
6. Morgan → บันทึก Log การเรียก API ลงคอนโซล
7. Static → ให้บริการไฟล์รูปภาพใน /uploads
8. Routing → ลงทะเบียน API 8 เส้นทางหลัก
9. Health → GET /health ตรวจสอบสถานะเซิร์ฟเวอร์
10. 404 → ดักจับ URL ที่ไม่มีในระบบ
11. Listen → เปิดรับการเชื่อมต่อ Port 5000

🚩 โค้ดตัวอย่างสำคัญ — การลงทะเบียน API Routes

```
// — ลงทะเบียนเส้นทางหลักของ API —
app.use('/api/auth', require('./routes/auth.routes')); // เข้าสู่ระบบ, Token
app.use('/api/master', require('./routes/master.routes')); // Master Data 9 หมวด
app.use('/api/projects', require('./routes/project.routes')); // CRUD โครงการ, Lock
app.use('/api/activities', require('./routes/activity.routes')); // กิจกรรม, อัปเดตรูป
app.use('/api/dashboard', require('./routes/dashboard.routes')); // แดชบอร์ดตาม Role
app.use('/api/directives', require('./routes/directive.routes')); // ข้อสั่งการผู้บริหาร
app.use('/api/reports', require('./routes/report.routes')); // รายงานสรุป
app.use('/api/issues', require('./routes/issue.routes')); // แจ้งปัญหาระบบ
```

🚩 โค้ดสำคัญ — ระบบป้องกัน Brute-force Login

```
// ป้องกันการล็อกอินกระหน่ำ: อนุญาตแค่ 20 ครั้ง ต่อ 15 นาที ต่อ 1 IP
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 นาที
  max: 20,
  message: { message: 'ท่านทำธุรกรรมบ่อยเกินไป กรุณาลองใหม่ใน 15 นาที' }
});
app.use('/api/auth/login', authLimiter);
```

2. 🛒 ระบบความปลอดภัย — `backend/middleware/auth.middleware.js`

หน้าที่: ด้านตรวจสอบ 2 ชั้น ที่คำขอ API ทุกตัวต้องผ่าน ก่อนเข้าถึงข้อมูลในระบบ

ขั้นที่ 1: `authenticate` — ตรวจสอบตัวตน (JWT Verification)

```
const authenticate = async (req, res, next) => {
  // — 1. ตรวจสอบ Authorization Header —————
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer '))
    return res.status(401).json({ message: 'Authorization token required' });

  // — 2. ถอดรหัส Token เพื่อหา User ID —————
  const token = authHeader.split(' ')[1];
  const decoded = jwt.decode(token);

  // — 3. ดึงข้อมูลผู้ใช้จาก Database (พร้อมสังกัดคณะ/ภาควิชา) —————
  const user = await prisma.user.findUnique({
    where: { id: decoded.id },
    include: { department: { include: { faculty: true } } }
  });

  // — 4. ✨ เทคนิคพิเศษ: ผูก Secret กับ Password Hash —————
  // เมื่อเปลี่ยนรหัสผ่าน → Token เดิมทั้งหมดจะหมดอายุทันทีอัตโนมัติ!
  jwt.verify(token, process.env.JWT_SECRET + user.password);

  req.user = user; // แแนบข้อมูลผู้ใช้ให้ Controller ใช้ต่อ
  next();
};
```

ขั้นที่ 2: `authorize` — ตรวจสอบสิทธิ์ตามบทบาท (RBAC)

```
// ตรวจสอบว่า Role ของผู้ใช้ตรงกับบทบาทที่อนุญาตหรือไม่
const authorize = (roles = []) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Forbidden: Insufficient privileges' });
    }
    next();
  };
};

// ตัวอย่างการใช้งานใน Routes:
router.get('/dean-dashboard', authenticate, authorize(['DEAN', 'ADMIN']), getDeanStats);
router.get('/president-dashboard', authenticate, authorize(['PRESIDENT', 'ADMIN']), getPresidentStats);
```

สถานะ	ความหมาย
✅ 200 OK	ผ่านทั้ง 2 ด้าน เข้าถึงข้อมูลได้
❌ 401 Unauthorized	ไม่มี Token หรือ Token ผิด/หมดอายุ
❌ 403 Forbidden	มี Token ถูกต้อง แต่ Role ไม่มีสิทธิ์

3. 📁 โครงสร้างฐานข้อมูล — `backend/prisma/schema.prisma`

หน้าที่: แบบจำลองโครงสร้างข้อมูลทั้ง 14 ตาราง พร้อมความสัมพันธ์และกฎความปลอดภัย

โครงสร้างตารางหลัก (14 Tables)

📦 Master Data (ข้อมูลพื้นฐาน)	
— faculties	— คณะวิชา
— departments	— ภาควิชา / สาขาวิชา
— fiscal_years	— ปีงบประมาณ (เปิด/ปิด Active)
— budget_sources	— แหล่งที่มาของงบประมาณ
— local_development_issues	— ประเด็นการพัฒนาท้องถิ่น (ระดับ 1)
— strategies	— ยุทธศาสตร์มหาวิทยาลัย (ระดับ 2)
— sub_strategies	— แผนงานย่อย / กลยุทธ์ (ระดับ 3)
— indicators	— โครงการหลัก / ตัวชี้วัด KPI (ระดับ 4)
👤 User & Auth	
— users	— บัญชีผู้ใช้งาน (Role: ADMIN, TEACHER, DEAN, PRESIDENT)
🔄 Project Core	
— projects	— ข้อเสนอโครงการ (หัวใจหลักของระบบ)
— project_users	— ผู้รับผิดชอบร่วมโครงการ (Junction Table)
— activities	— กิจกรรมย่อยในโครงการ
— activity_images	— รูปภาพหลักฐานกิจกรรม
— issue_reports	— รายงานปัญหาการใช้งาน

🔗 โค้ดโมเดลสำคัญ — ตาราง `Project`

```
model Project {
  id          Int          @id @default(autoincrement())
  name        String       // ชื่อโครงการ
  totalBudget Float        // งบประมาณรวมที่ได้รับอนุมัติ
  usedBudget  Float        @default(0) // งบที่ใช้จริงสะสม
  targetCount Int          // เป้าหมายผลผลิตรวม
  completedCount Int       @default(0) // ผลผลิตที่ทำได้จริง
  remainingCount Int       @default(0) // ผลผลิตคงเหลือ
  progress    Float        @default(0) // % ความก้าวหน้าสะสม
  isLocked    Boolean      @default(false) // 🔒 ล็อกแผนงาน
  startDate   DateTime
  endDate     DateTime
  // — ความสัมพันธ์ยุทธศาสตร์ 4 ชั้น —
  subStrategyId Int        // → sub_strategies
  indicatorId    Int?       // → indicators (KPI)
  fiscalYearId   Int        // → fiscal_years
  budgetSourceId Int        // → budget_sources
  facultyId      Int?       // → faculties
  departmentId   Int?       // → departments
  creatorId      Int        // → users (ผู้สร้าง)
  // — ข้อมูลการผู้บริหาร —
  deanDirective String?     @db.Text // คำสั่งจากคณบดี
  presDirective  String?    @db.Text // คำสั่งจากอธิการบดี
  activities      Activity[]
  users           ProjectUser[]
}
```

4. 📁 ตรรกะธุรกิจหลัก — `backend/controllers/project.controller.js`

หน้าที่: Logic หลักของระบบ ครอบคลุมการสร้างโครงการ, คำนวณความก้าวหน้า และระบบล็อกแผนงาน

🚩 พังกัชั้นคำนวณความก้าวหน้า (Progress Rollup)

```
// — เรียกทุกครั้งหลังบันทึกกิจกรรมใหม่ → คำนวณ % ของโครงการหลักอัตโนมัติ —
const updateProjectProgress = async (projectId, txClient = prisma) => {
  const project = await txClient.project.findUnique({
    where: { id: projectId },
    include: { activities: true } // ดึงกิจกรรมย่อยทั้งหมด
  });

  // รวมยอดผลผลิตจากทุกกิจกรรมย่อย (Activity Rollup)
  const completedCount = project.activities.reduce((sum, a) => {
    return sum + (a.completedCount > 0 ? a.completedCount : (a.success ? 1 : 0));
  }, 0);

  // คำนวณ % ความก้าวหน้า (สูงสุดไม่เกิน 100%)
  const progress = Math.min(
    parseFloat(((completedCount / project.targetCount) * 100).toFixed(2)),
    100.0
  );

  // อัปเดตกลับสู่ตาราง Project ทันที (Real-time Progress)
  await txClient.project.update({
    where: { id: projectId },
    data: { completedCount, remainingCount, progress }
  });
};
```

🚩 การสร้างโครงการด้วย Database Transaction

```
// ใช้ Transaction เพื่อให้มั่นใจว่าการบันทึกสำเร็จครบถ้วนหรือไม่สำเร็จเลย
const project = await prisma.$transaction(async (tx) => {
  // 1. สร้างโครงการหลักพร้อมเชื่อมโยงยุทธศาสตร์ 4 ระดับ
  const proj = await tx.project.create({
    data: { name, totalBudget, targetCount, subStrategyId, fiscalYearId, ... }
  });

  // 2. ผู้รับผิดชอบร่วมโครงการ (Project Team Members)
  if (userIds && userIds.length > 0) {
    await tx.projectUser.createMany({
      data: userIds.map(uid => ({ projectId: proj.id, userId: uid }))
    });
  }
  return proj;
});
```

5. 📊 แดชบอร์ดตามบทบาท — `backend/controllers/dashboard.controller.js`

หน้าที่: คำนวณและส่งสถิติแดชบอร์ดที่แตกต่างกันตาม Role ของผู้ใช้งาน

ระบบตรวจจับโครงการวิกฤต (Red Flag Detection – Management by Exception)

```
// ——— คำนวณสัญญาณไฟ RAG (Red / Amber / Green) ———
const calculateProjectRAG = (project) => {
  const now = new Date();
  const start = new Date(project.startDate);
  const end = new Date(project.endDate);

  const totalDays = (end - start) / (1000 * 60 * 60 * 24);
  const elapsed = (now - start) / (1000 * 60 * 60 * 24);
  const timeProgress = (elapsed / totalDays) * 100; // % เวลาที่ผ่านไป

  // 🔴 RED: หมดเวลาแล้ว แต่งานไม่เสร็จ / หรือ เวลาผ่านไป >50% แต่งาน <25%
  if (
    (now > end && project.progress < 100) ||
    (timeProgress > 50 && project.progress < 25)
  ) return 'RED';

  // 🟡 AMBER: ใกล้ครบกำหนด หรือ ความก้าวหน้าน้อยกว่าเวลา >20%
  if (timeProgress - project.progress > 20) return 'AMBER';

  // 🟢 GREEN: ปกติ
  return 'GREEN';
};
```

API แดชบอร์ดแยกตาม Role

Endpoint	Role	สิ่งที่แสดง
GET /api/dashboard	TEACHER / ADMIN	โครงการที่ตนดูแล, กิจกรรม, รูปภาพล่าสุด
GET /api/dashboard/dean	DEAN	ภาพรวมคณะ, Cross-Dept Matrix, Red Flags ของคณะ
GET /api/dashboard/president	PRESIDENT	Strategic Heatmap, Red Flags ทั้งมหาวิทยาลัย

6. 🏠 หน้าบ้าน (Frontend) — `frontend/src/App.jsx`

หน้าที่: จัดการ Routing หน้าจอทั้งหมด สลับหน้าจออัตโนมัติตามบทบาทผู้ใช้

ระบบ Lazy Loading (โหลดหน้าจอเมื่อต้องการ)

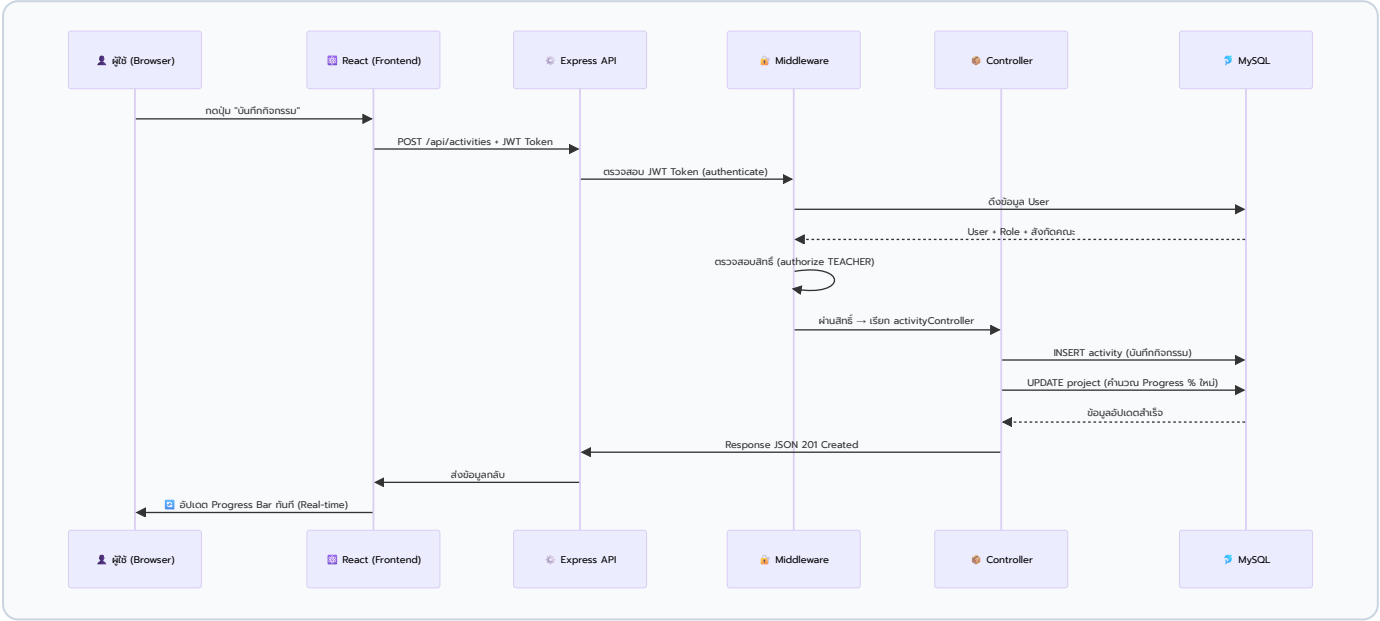
```
// ——— โหลดหน้าจอตาม Role แบบ Lazy (ประหยัด Memory) ———
const TeacherDashboard = lazy(() => import('./pages/teacher/Dashboard'));
const DeanDashboard = lazy(() => import('./pages/dean/Dashboard'));
const PresidentDashboard = lazy(() => import('./pages/president/Dashboard'));
const AdminDashboard = lazy(() => import('./pages/admin/Dashboard'));

// ——— ตัวสลับหน้าแดชบอร์ดอัตโนมัติตาม Role ———
const DashboardResolver = () => {
  const { user } = useContext(AuthContext);
  switch (user?.role) {
    case 'ADMIN': return <AdminDashboard />;
    case 'DEAN': return <DeanDashboard />;
    case 'PRESIDENT': return <PresidentDashboard />;
    default: return <TeacherDashboard />;
  }
};
```

ระบบป้องกันเส้นทาง (Protected Route Guard)

```
// — ถ้าไม่ได้ล็อกอิน จะถูก Redirect กลับไปหน้า /login แทนที่ —
const ProtectedRoute = ({ children }) => {
  const { user, loading } = useContext(AuthContext);
  if (loading) return <PageLoader />;
  if (!user) return <Navigate to="/login" replace />;
  return children;
};
```

7. ลำดับการทำงานจริงตั้งแต่ต้นถึงปลาย (End-to-End Flow)



8. ไพล์สำคัญในระบบ (Quick Reference Map)

ไพล์	หน้าที่ 1 ประโยค
backend/app.js	จุดเริ่มต้นเซิร์ฟเวอร์ ตั้งค่าความปลอดภัยและ Routes
middleware/auth.middleware.js	ตรวจสอบ JWT + คู่มือสิทธิ์ตาม Role (RBAC)
controllers/project.controller.js	Logic หลัก: สร้างโครงการ, คำนวณ % Progress, ล็อกแผน
controllers/dashboard.controller.js	คำนวณสถิติแดชบอร์ดและตรวจจับ Red Flags
controllers/auth.controller.js	ล็อกอิน, ออก JWT Token, เปลี่ยนรหัสผ่าน
prisma/schema.prisma	แบบจำลองฐานข้อมูล 14 ตารางและความสัมพันธ์
frontend/src/App.jsx	Routing หน้าบ้านและสลับ Dashboard ตาม Role

💡 เทคนิคการนำเสนอ:
อธิบายตามลำดับ: ฐานข้อมูล (Schema) → ความปลอดภัย (Middleware) → Logic หลัก (Controller) → หน้าจอ (React)
เพื่อให้กรรมการเห็นการไหลของข้อมูลตั้งแต่ต้นน้ำถึงปลายน้ำครับ